\

PERTEMUAN 4: *ARRAY*, *STRING*, DAN *STRUCTURE*

4.1. Tujuan

1. Mahasiswa memahami konsep array sebagai struktur data untuk menyimpan kumpulan elemen bertipe sama.
2. Mahasiswa memahami konsep string sebagai kumpulan karakter dan implementasinya di C++.
3. Mahasiswa mampu mendeklarasikan, menginisialisasi, dan mengakses elemen array (satu dimensi dan multidimensi).
4. Mahasiswa mampu mengimplementasikan *array* dan *string* untuk menyelesaikan masalah pemrograman sederhana.
5. Mahasiswa memahami bagaimana mengelompokkan beberapa variabel dengan tipe data yang berbeda menggunakan *structure*.

4.2. Dasar Teori

Tidak jarang ketika membuat program, kita perlu menyimpan banyak data yang sejenis. Sebagai analogi, bayangkan Anda ingin mencatat nilai ujian untuk 30 mahasiswa di satu kelas atau menyimpan daftar nama tamu undangan. Alih-alih membuat 30 variabel terpisah (misalnya `nilai1`, `nilai2`, `nilai3`, ...), akan jauh lebih efisien jika kita bisa menyimpannya dalam satu 'wadah' yang terorganisir. Dalam konteks pemrograman, 'wadah' ini disebut sebagai *Array*. *Array* adalah struktur data fundamental yang memungkinkan kita menyimpan sekumpulan elemen dengan tipe data yang sama dalam satu blok memori yang berurutan. Setiap elemen data dalam *array* dapat diidentifikasi dan diakses secara unik menggunakan sebuah *indeks* (nomor posisi).

Salah satu penerapan khusus dari array adalah untuk menyimpan data teks. Kumpulan karakter (seperti 'H', 'a', 'l', 'o') yang membentuk sebuah kata atau kalimat dikenal sebagai *string*. Di C++, *string* ditangani sebagai *array* dari tipe data `char` (dikenal sebagai C-style string). Namun, C++ modern juga menyediakan *library string* yang lebih aman dan kaya fitur untuk memanipulasi teks.

Modul ini akan membahas cara kerja *array* (baik satu dimensi maupun multidimensi), dua pendekatan untuk penanganan string di C++, dan pembuatan *structure*.

4.2.1. Deklarasi dan inisialisasi *array*

Deklarasi *array* dilakukan untuk memberitahu *compiler* bahwa program akan menggunakan suatu *array* dengan tipe data dan jumlah elemen tertentu. Bentuk umum deklarasi array adalah sebagai berikut :

```
tipe_data nama_array[jumlah_elemen];
```

Fungsi dari sintaks diatas adalah:

1. `tipe_data` menunjukkan jenis data yang akan disimpan (`int`, `float`, `char`, `string`)
2. `nama_array` merupakan nama variabel dari array yang dibuat
3. `jumlah_elemen` menunjukkan banyaknya anggota di dalam array tersebut

Contoh deklarasi *array*:

```
int wadah_buah[10];
```

Artinya, dibuat sebuah *array* dengan tipe jenis data `int`, dan dapat diisi dengan 10 elemen. Inisiasi *array* merupakan sebuah proses mengisi nilai awal ke dalam elemen-elemen *array*. Inisialisasi dapat dilakukan dengan beberapa cara, antar lain

1. Inisiasi saat deklarasi :

   ```
   int nilai[3] = {7, 7, 7};
   ```

   Artinya setiap elemen *array*, sudah langsung diisi dengan angka-angka

2. Inisiasi tanpa menentukan jumlah elemen

   ```
   int angka[] = {4, 4, 4};
   ```

   Compiler otomatis akan menentukan ukuran *array* berdasarkan jumlah nilai yang diberikan (dalam contoh, ini berukuran 3).

4.2.2. Mengakses elemen *array*

Mengakses sebuah elemen *array* dapat dilakukan dengan cara menggunakan metode *indexing*, dimulai dari 0. Contoh:

```
int angka[] = { 1, 4, 5 };

cout << angka[0];
cout << angka[1];
```

4.2.3. Array dan perulangan (looping)

Selain melakukan *indexing*, akses array juga bisa digabungkan dengan menggunakan metode for loop / perulangan agar lebih mudah dan cepat;

```
for ( int i = 0; i < 5; i++ ) {
    cout << “Nilai Ke :” << i << “ =” << angka[i] << endl;
```

Di dalam perulangan tersebut, indeks `i` digunakan untuk mengakses setiap elemen array berdasarkan urutannya. Hasilnya:

```
Nilai Ke : 0 = 1
Nilai Ke : 1 = 4
Nilai Ke : 2 = 5
```

4.2.4. Array dan percabangan

Array dan percabangan merupakan sebuah gabungan metode antara struktur data *array* dengan struktur logic `if - else`. Hal ini dilakukan untuk pengambilan keputusan berdasarkan nilai yang tersimpan dalam *array*. Kombinasi ini digunakan ketika membuat sebuah program untuk menentukan nilai maksimum, ganjil genap, ataupun mengelompokkan data. Contoh menentukan bilangan ganjil / genap:

```
#include <iostream>
using namespace std;

int main() {
    int angka[5] =  {1, 3, 2, 5, 8} ;
    for ( int i = 0; i < 5; i++ ) {
          if ( angka[i] % 2 != 0 ) {
                cout << angka[i] << “ Angka Ganjil” << endl ; }
          else {
                cout << angka[i] << “ Angka Genap” << endl ; }
          }
    return 0 ;
}
```

Blok kode di dalam `for loop` akan terus dijalankan selama masih memenuhi syarat yakni `i < 5` , dan sistem akan mengeksekusi *statement* di dalam nya. Yang dimana jika angka[i] tidak habis dibagi 2 maka ganjil, dan sebaliknya. Hasil Output:

```
1 Angka Ganjil
3 Angka Ganjil
2 Angka Genap
5 Angka Ganjil
8 Angka Genap
```

4.2.5. Array bertipe karakter

Merupakan sebuah *array* yang dimana isinya berupa tipe data karakter (`char`) biasanya *array* ini sering digunakan untuk menyimpan sebuah kata atau kalimat. Setiap elemen dari *array* ini bisa diakses dengan *indexing*, namun diakhiri dengan karakter khusus ‘\0’ (null character) yang menandakan akhir dari *string*

1. Contoh deklarasi dan inisiasi *array*:

```
#include <iostream>
using namespace std;

int main() {
    char nama1[4] = {‘C’, ‘A’, ‘T’, ‘\0’} ; // Inisiasi manual
    char nama2[] = “DOG”           ; // Inisiasi otomatis

    cout << “NAMA 1 : “ << nama1 << endl ;
    cout << “NAMA 2 : “ << nama2 << endl ;
    return 0;
}
```

Hasil output :

```
Nama 1 : CAT
Nama 2 : DOG
```

2. Contoh akses tiap karakter:

```
#include <iostream>
using namespace std;

int main() {
    char nama[] = ‘FLY’ ;
    for ( int i=0 ; nama[i] != ‘\0’; i++ ) {
          cout << “Isi Data : “ << nama[i] << endl ;
    }
    return 0;
}
```

Hasil output :

```
Isi Data : F
Isi Data : L
Isi Data : Y
```

3. Contoh input string pada array:

```
#include <iostream>
using namespace std;

int main() {
    char nama[30] ;

    cout << “Masukkan Nama Anda : “ << endl ;
    cin.getline(nama, 30) ;

    cout << “Hallo “ << nama << “!” << endl ;
    return 0 ;
}
```

Hasil output :

```
Masukkan Nama Anda : Kevin Arduino
Hallo Kevin Arduino!
```

4.2.6. Tipe data string

C++ menyediakan *library* modern yang jauh lebih aman dan mudah digunakan untuk manipulasi teks, yaitu `std::string`. Tipe data ini adalah bagian dari *Standard Template Library* (STL) C++ dan memerlukan `include <string>` untuk menggunakannya. Berbeda dengan array `char`, `string` adalah sebuah *class* yang mengelola memorinya secara otomatis. Ukuran *string* dapat bertambah atau berkurang secara dinamis tanpa perlu kita atur secara manual. Ini menghilangkan risiko *buffer overflow* yang sering terjadi pada

```
#include <iostream>
#include <string> // Diperlukan untuk menggunakan std::string ...
using namespace std;

int main() {
    // Deklarasi string kosong
    string sapaan;

    // Inisialisasi langsung
    string nama = "Budi";

    // Inisialisasi dengan salinan
    string nama_lain = nama;

    return 0 ;
}
```

4.2.7. Tipe data string

Kelebihan utama *string* adalah banyaknya fungsi dan fitur bawaan yang memudahkan operasi pada teks. Berikut adalah beberapa operasi dasar yang paling sering digunakan:

1. Penggabungan (*Concatenation*): Menggabungkan dua string atau lebih menggunakan operator '+'.

```
    string sapaan = “Halo, ”;
    string nama = “Budi.”;

    string kalimat = sapaan + nama; //
    return 0 ;
}
```

2. Mendapatkan panjang: Menggunakan fungsi `.length()` atau `.size()` untuk mengetahui jumlah karakter di dalam string.

```
string kata = “Informatika”;
int panjang  = kata.length();   // 10
```

3. Mengakses Karakter: Sama seperti *array,* karakter individu dapat diakses menggunakan *indeks* (dimulai dari 0) dengan operator `[]` atau fungsi `.at()`.

```
string kata = “C++”;
char huruf_pertama = kata[0]         // Output : ‘C’;
char huruf_kedua =  kata.at(1);      // Output : ‘+’;
```

4. Input kalimat**:** Untuk membaca input string dari pengguna, `cin` dapat digunakan. Namun, `cin` akan berhenti membaca ketika bertemu spasi. Untuk membaca satu baris penuh (termasuk spasi), gunakan fungsi `getline()`.

```
string nama;
cout << “Masukkan namamu: ”;
cin >> nama;
    // Jika menginputkan ‘Andre Patrick’, yang tersimpan hanya ‘Andre’

string nama_lengkap;
cout << “Masukkan namamu: ”;  // Output : ‘C’;
getline(cin, nama_lengkap);
    // Jika menginputkan ‘Andre Patrick’, yang tersimpan adalah ‘Andre Patrick’
```

5. Perbandingan**:** String dapat dibandingkan berdasarkan urutan alfabet menggunakan operator perbandingan standar (==, !=, <, >).

```
string a = “Apel”;
string b = “Jeruk”;

cout << (a < b);  // Output : 1
    // Kondisi ini bernilai true karena ‘A’ dan ‘J’ secara urutan lebih besar ‘J’.
```

4.2.8. Tipe data string

Array Multidimensi merupakan sebuah *array* yang berada di dalam *array*. *Array* ini dapat dideklarasikan dengan menambah `[]` menjadi 2 atau lebih dan mengisi masing-masing berapa ukuran yang akan ditampung. Inisialisasi array 2D dapat dilakukan saat deklarasi menggunakan kurung kurawal bersarang. Berikut adalah contoh deklarasi *array* 2D untuk menyimpan matriks 3x4 (3 baris, 4 kolom).

```
int matriks[3][4][ = {
      {1, 2, 3, 4}, // Baris ke-0
      {5, 6, 7, 8}, // Baris ke-1
```

```
      {9, 10, 11, 12} // Baris ke-2
};
```

Untuk mengakses elemen tertentu dalam array 2D, kita memerlukan dua indeks: satu untuk baris dan satu untuk kolom.

```
cout << matriks[0][1] << endl;   // Output : 2
cout << matriks[1][2] << endl;   // Output : 7
cout << matriks[2][3] << endl;   // Output : 12
```

Untuk mengakses seluruh elemen pada *array* multidimensi dapat dilakukan menggunakan *nested looping*.

```
for (int i = 0; i < 3; i++) {
       for (int j = 0; j < 4; j++) p{
            cout << matriks[i][j] << " ";
       }
       cout << endl;
}
```

4.2.9. *Structure*

Structure atau biasa disebut dengan *struct* adalah metode dalam C++ untuk mengelompokkan beberapa variabel yang saling terkait. Variabel-variabel ini disebut member dari sebuah *struct*. *Struct* dapat diisi oleh beberapa variabel dengan tipe data yang berbeda. Kode di bawah ini mendemonstrasikan cara mendeklarasikan *struct*.

```
struct Mahasiswa{
      string nama;
      int umur;
      float IPK;
};
```

Pada kode di atas, struct Mahasiswa dideklarasikan memiliki tiga member yaitu nama dengan tipe string, umur dengan tipe int, dan IPK dengan tipe float. Untuk mengakses dan menugaskan masing-masing member dapat dilakukan dengan cara berikut,

```
Mahasiswa a;

a.nama = "Rudi";
a.umur = 22;
a.IPK = 3.7;
```

4.3. Aktivitas Praktikum

1. Buatlah program yang berfungsi sebagai pencatat jadwal mata kuliah sederhana untuk 5 hari kerja (Senin-Jumat) dengan menggunakan array. Dimana jadwal mata kuliahnya sebagai berikut :
   a. Senin : Matematika Dasar
   a. Selasa : Literasi Digital
   a. Rabu : Penjasbug
   a. Kamis : Matematika Diskrit
   a. Jumat : Pemrograman Dasar

   Program meminta pengguna memasukkan sebuah angka (1-5) untuk melihat jadwal hari tersebut (1=Senin, 5=Jumat). Program akan menampilkan jadwal untuk hari yang dipilih. Contoh output: "Jadwal hari [Nama Hari]: [Nama Mata Kuliah]". Jika input di luar angka 1-5, maka terdapat pemberitahuan “Input hari tidak valid”.

2. Buatlah program yang meminta pengguna memasukkan sebuah kata sandi. Program akan memvalidasi kata sandi tersebut berdasarkan dua aturan menggunakan operasi string
   A. Aturan 1: Kata sandi harus sama persis dengan ‘AMANda123’
   A. Aturan 2: Kata sandi minimal harus memiliki 8 karakter.

   Program harus memberikan luaran sebagai berikut:
   A. Jika kata sandi benar, cetak "Login berhasil.".
   A. Jika kata sandi salah tetapi panjangnya kurang dari 8 karakter, cetak "Kata sandi salah dan terlalu pendek (minimal 8 karakter).".
   A. Jika kata sandi salah tetapi panjangnya 8 karakter atau lebih, cetak "Kata sandi salah.".
3. Buatlah program untuk merekap nilai mahasiswa dengan ketentuan berikut:
   1. Definisikan struct Mahasiswa yang berisi nim (string), nama (string), nilai (array float berisi 3 elemen untuk Tugas, UTS, UAS), dan rataRata (float).
   2. Simpan data maksimal 10 mahasiswa dalam array of struct. Jumlah mahasiswa dimasukkan oleh pengguna.
   3. Untuk setiap mahasiswa, input NIM, nama (boleh mengandung spasi), dan tiga nilai.
   4. Hitung rata-rata nilai setiap mahasiswa.
   5. Tampilkan seluruh data dalam bentuk tabel, dengan nama ditulis dalam huruf kapital.
   6. Tampilkan nama mahasiswa dengan rata-rata tertinggi.

4.4. Tugas Praktikum

1. Program Studi S1 Kecerdasan Artifisial Unesa ingin membuat program sederhana untuk menampilkan daftar nama mahasiswa di suatu kelas. Data nama mahasiswa disimpan dalam bentuk array string. Program ini harus menampilkan daftar nama mahasiswa berdasarkan urutan indeksnya. Buat sebuah program array string dengan ukuran 5 elemen untuk menampung nama mahasiswa.Minta pengguna memasukkan nama-nama mahasiswa satu per satu melalui keyboard. Setelah semua nama dimasukkan, tampilkan seluruh nama mahasiswa dengan format yang rapi (menampilkan nomor urut dan nama).

2. Sebuah panitia lomba 17 Agustus menyimpan daftar nama peserta dalam sebuah array string. Panitia ingin memiliki program untuk mencari apakah nama tertentu terdaftar atau tidak dalam daftar peserta. Program harus menanyakan kepada pengguna nama yang ingin dicari, lalu menampilkan hasil apakah nama tersebut ditemukan atau tidak. Buat program array string berisi daftar 10 nama peserta yang sudah ditentukan di dalam program (tidak perlu input manual).Lalu,minta pengguna memasukkan nama yang ingin dicari. Program melakukan pencarian nama dengan perbandingan string (case-sensitive). Jika nama ditemukan, tampilkan posisi indeksnya di array; jika tidak, tampilkan pesan bahwa nama tidak ditemukan.

3. Seorang penulis ingin menganalisis kalimat yang ia tulis dengan melihat berapa banyak kata yang terdapat dalam kalimat tersebut, serta menampilkan daftar katanya secara terpisah. Program ini harus mampu menerima satu kalimat utuh dari pengguna, memecahnya menjadi array string per kata, menghitung jumlah katanya, dan menampilkan daftar kata beserta indeksnya. Buat program array string untuk minta pengguna untuk memasukkan satu kalimat utuh (contoh: "Belajar C++ itu menyenangkan dan bermanfaat"). Lalu, pisahkan kalimat menjadi kata-kata, lalu simpan setiap kata ke dalam array string (gunakan metode manual dengan stringstream atau loop berdasarkan spasi). Hitung jumlah total kata yang ditemukan.Tampilkan juga daftar kata beserta indeksnya, serta total jumlah kata.

4. Sebuah perpustakaan kecil membutuhkan program untuk mengelola data bukunya. Buatlah program dengan ketentuan berikut:

    a. Definisikan struct `Buku` yang berisi `kode` (`string`), `judul` (`string`), `penulis` (`string`), `tahun` (`int`), dan `stok` (`int`).

    b. Data disimpan dalam array of struct berkapasitas maksimal 50 buku.

    c. Program menampilkan menu berulang hingga pengguna memilih keluar:

1. Tambah buku
2. Tampilkan semua buku
3. Cari buku berdasarkan kata kunci judul
4. Urutkan buku berdasarkan judul (A–Z)
5. Keluar

d. Validasi kode buku: kode harus berformat BK- diikuti tepat 3 digit angka (contoh: BK-007). Jika format salah, tampilkan pesan kesalahan dan minta input ulang. Kode juga tidak boleh sama dengan kode buku yang sudah ada.

e. Pencarian judul tidak membedakan huruf besar dan kecil dan dapat mencocokkan sebagian judul. Contoh: kata kunci pemrograman harus menemukan buku berjudul Dasar Pemrograman C++.

f. Pengurutan dilakukan secara manual (misalnya *bubble sort* atau *selection sort*), tanpa menggunakan `sort()` dari STL.

g. Pada tampilan semua buku, tampilkan juga total stok seluruh buku.